

C++ Standard Library Issues to be moved in Hagenberg, Feb. 2025

Doc. no. P3615R0

Date: 2025-02-07

Audience: WG21

Reply to: Jonathan Wakely <lwgchair@gmail.com>

Tentatively Ready Issues

[3578](#). Iterator SCARYness in the context of associative container merging

Section: 23.2.7.1 [\[associative.reqmts.general\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Joaquín M López Muñoz **Opened:** 2021-08-04 **Last modified:** 2024-12-09

Priority: 3

View other [active issues](#) in [\[associative.reqmts.general\]](#).

View all other [issues](#) in [\[associative.reqmts.general\]](#).

Discussion:

For the expression `a.merge(a2)`, postconditions say that "iterators referring to the transferred elements [...] now behave as iterators into a [...]". When `a` and `a2` are of different types, this seems to imply, under the widest interpretation for "behaving as", that `a`-iterators and `a2`-iterators are actually of the same type, that is, that associative containers have SCARY iterators, which is, to the best of my knowledge, not currently mandated by the standard.

There are (at least) three possible resolutions to this ambiguity, ordered by intrusiveness:

- Indicate that "behaving as" only applies to the case where `a` and `a2` are of the same type.
- Clarify what "behaving as" means. A non-SCARY interpretation is that an `a2`-iterator to a transferred element can still be dereferenced, incremented (if not past the last element of `a`) and decremented (if not pointing to the first element of `a`), while comparison with `a`-iterators and use in the interface of `a` is not guaranteed.
- Mandate SCARY iterators by, for instance, requiring that associative containers with compatible nodes (23.2.5.1 [\[container.node.overview\]](#)/1) have the same iterator types.

Note that this issue does not extend to unordered associative containers, as there (23.2.8.1 [\[unord.req.general\]](#)) iterators to transferred elements are invalidated, which makes the point of SCARYness irrelevant. That said, if SCARY iterators are finally required for associative containers, it makes much sense to extend the requirement to unordered associative containers as well.

[2021-08-20; Reflector poll]

Set priority to 3 after reflector poll.

[2024-12-04; Jonathan provides wording]

If we want to require [SCARY](#) iterators then that should be a proposal that goes through LEWG design review. I propose an almost minimal change to make the spec consistent without imposing any new requirements on implementations.

The minimal change would be to say that iterators remain valid if `a` and `a2` have the same type, which is the minimum portable guarantee that always holds. However what matters in practice is whether the iterator types are the same. That would not be a portable guarantee, because it depends on whether the implementation uses SCARY iterators for its maps and sets, so users could write code that works on one implementation and fails to compile when moved to a different implementation. But that portability trap would be present even if we only say iterators remain valid when `a` and `a2` are the same type. If the code compiles and works on an implementation with SCARY iterators, then users will rely on that, even if unintentionally. Leaving that case unspecified or undefined in the standard doesn't prevent users from relying on it. It doesn't seem to serve any benefit to pretend it doesn't work when it actually does on some implementations.

N.B. Libstdc++ associative container iterators are SCARY by default, but non-SCARY when `-D_GLIBCXX_DEBUG` is used to enable Debug Mode (see [Bug 62169](#)). I believe libc++ iterators are SCARY even when `-D_LIBCPP_HARDENING_MODE=_LIBCPP_HARDENING_MODE_DEBUG` is used, and MSVC STL iterators are SCARY even when `/D_ITERATOR_DEBUG_LEVEL` is used.

[2024-12-09; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4993](#).

1. Modify 23.2.7.1 [\[associative.reqmts.general\]](#) as indicated:

a.merge(a2)

-112- *Result*: void

-113- *Preconditions*: a.get_allocator() == a2.get_allocator() **is true**.

-114- *Effects*: Attempts to extract each element in a2 and insert it into a using the comparison object of a. In containers with unique keys, if there is an element in a with key equivalent to the key of an element from a2, then that element is not extracted from a2.

-115- *Postconditions*: Pointers and references to the transferred elements of a2 refer to those same elements but as members of a. **If a.begin() and a2.begin() have the same type, iterators** ~~Iterators~~ referring to the transferred elements will continue to refer to their elements, but they now behave as iterators into a, not into a2.

-116- *Throws*: Nothing unless the comparison objects throws.

-117- *Complexity*: $N \log(a.size() + N)$, where N has the value a2.size().

3956. chrono::parse uses from_stream as a customization point

Section: 30.13 [\[time.parse\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jonathan Wakely **Opened:** 2023-07-15 **Last modified:** 2024-12-09

Priority: 3

View other [active issues](#) in [time.parse].

View all other [issues](#) in [time.parse].

Discussion:

30.13 [\[time.parse\]](#) says: "Each parse overload specified in this subclause calls from_stream unqualified, so as to enable argument dependent lookup (6.5.4 [\[basic.lookup.argdep\]](#))." That name should be added to 16.4.2.2 [\[contents\]](#) along with swap, make_error_code, and make_error_condition.

We should decide whether calls to from_stream should use normal lookup (i.e. unqualified lookup plus ADL) or just ADL, as was done for make_error_code and make_error_condition (see LWG [3629\(i\)](#)).

[2023-10-30; Reflector poll]

Set priority to 3 after reflector poll.

[2024-12-02; Jonathan provides wording]

I suggest that from_stream should only be found via ADL, not unqualified lookup. This is consistent with what we did for make_error_code and make_error_condition, and more recently for submdspan_mapping. I see no reason to treat from_stream differently. This implies that implementations might need a poison poll in std::chrono so that unqualified lookup stops as soon as those are found.

[2024-12-09; Reflector poll]

Set status to Tentatively Ready after six votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4993](#).

1. Modify 16.4.2.2 [\[contents\]](#) as indicated:

-3- Whenever an unqualified name other than swap, make_error_code, make_error_condition, **from_stream**, or submdspan_mapping is used in the specification of a declaration D in Clause 17 through Clause 33 or Annex D, its meaning is established as-if by performing unqualified name lookup (6.5.3 [\[basic.lookup.unqual\]](#)) in the context of D.

[Note 1: Argument-dependent lookup is not performed. — end note]

Similarly, the meaning of a *qualified-id* is established as-if by performing qualified name lookup (6.5.5 [\[basic.lookup.qual\]](#)) in the context of D.

[Example 1: The reference to `is_array_v` in the specification of `std::to_array` (23.3.3.6 [\[array.creation\]](#)) refers to `::std::is_array_v`. — end example]

[Note 2: Operators in expressions (12.2.2.3 [\[over.match.oper\]](#)) are not so constrained; see 16.4.6.4 [\[global.functions\]](#). — end note]

The meaning of the unqualified name `swap` is established in an overload resolution context for swappable values (16.4.4.3 [\[swappable.requirements\]](#)). The meanings of the unqualified names `make_error_code`, `make_error_condition`, `from_stream`, and `submdspan_mapping` are established as-if by performing argument-dependent lookup (6.5.4 [\[basic.lookup.argdep\]](#)).

4172. unique_lock self-move-assignment is broken

Section: 32.6.5.4.2 [\[thread.lock.unique.cons\]](#), 32.6.5.5.2 [\[thread.lock.shared.cons\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Casey Carter **Opened:** 2024-11-13 **Last modified:** 2025-02-07

Priority: Not Prioritized

View all other [issues](#) in [\[thread.lock.unique.cons\]](#).

Discussion:

The postconditions in 32.6.5.4.2 [\[thread.lock.unique.cons\]](#) paragraph 19:

Postconditions: `pm == u_p.pm` and `owns == u_p.owns` (where `u_p` is the state of `u` just prior to this construction), `u.pm == 0` and `u.owns == false`.

contradict themselves if `*this` and the parameter `u` refer to the same object. (Presumably "this construction" means the assignment, and it is copy-pasta from the move constructor postconditions.) Apparently `unique_lock` didn't get the memo that we require well-defined behavior from self-move-assignment as of LWG [2839^{\(1\)}](#).

Also, the move assignment operator doesn't specify what it returns.

[2024-11-18; Casey expands the PR to cover `shared_lock`]

`shared_lock` has the same problems, and can be fixed in the same way.

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

"Should use parentheses not braces for the initializations." Jonathan volunteers to do that editorially after this gets approved.

Proposed resolution:

This wording is relative to [N4993](#).

Drafting Note: I've chosen to use the move-into-temporary-and-swap idiom here to keep things short and sweet. Since move construction, swap, and destruction are all `noexcept`, I've promoted move assignment from "*Throws: Nothing*" to `noexcept` as well. This is consistent with eliminating the implicit narrow contract condition that `*this` and `u` refer to distinct objects.

1. In the class synopsis in 32.6.5.4.1 [\[thread.lock.unique.general\]](#), annotate the move assignment operator as `noexcept`:

```
namespace std {
    template<class Mutex>
    class unique_lock {
    [...]
        unique_lock& operator=(unique_lock&& u) noexcept;
    [...]
    };
}
```

2. Modify 32.6.5.4.2 [\[thread.lock.unique.cons\]](#) as follows:

`unique_lock& operator=(unique_lock&& u) noexcept;`

-18- *Effects:* ~~If `owns` calls `pm->unlock()`.~~ `Equivalent to: unique_lock{std::move(u)}.swap(*this).`

`-?- Returns: *this.`

~~-19- *Postconditions:* `pm == u_p.pm` and `owns == u_p.owns` (where `u_p` is the state of `u` just prior to this construction), `u.pm == 0` and `u.owns == false`.~~

~~-20- [Note 1: With a recursive mutex it is possible for both *this and u to own the same mutex before the assignment. In this case, *this will own the mutex after the assignment and u will not. — end note]~~

~~-21- Throws: Nothing.~~

3. Modify 32.6.5.5.2 [thread.lock.shared.cons] as follows:

shared_lock& operator=(shared_lock&& sl) noexcept;

~~-17- Effects: If owns calls pm->unlock_shared().~~ Equivalent to: shared_lock{std::move(sl)}.swap(*this).

~~-?- Returns: *this.~~

~~-18- Postconditions: pm == sl.p.pm and owns == sl.p.owns (where sl.p is the state of sl just prior to this assignment), sl.pm == nullptr and sl.owns == false.~~

4175. get_env() specified in terms of as_const() but this doesn't work with rvalue senders

Section: 33.5.2 [exec.get.allocator], 33.5.3 [exec.get.stop.token], 33.5.4 [exec.get.env], 33.5.5 [exec.get.domain], 33.5.6 [exec.get.scheduler], 33.5.7 [exec.get.delegation.scheduler], 33.5.8 [exec.get.fwd.progress], 33.5.9 [exec.get.compl.sched] **Status:** Tentatively Ready **Submitter:** Lewis Baker **Opened:** 2024-11-10 **Last modified:** 2025-02-07

Priority: Not Prioritized

Discussion:

The current specification of std::execution::get_env() defines get_env(o) as as_const(o).get_env(). However, the as_const() function has a deleted rvalue-taking overload, meaning that you cannot pass temporaries to it.

This means that several uses of get_env() which pass expressions which are either potentially rvalues (e.g. in definition of connect(sndr, rcvr) it uses the expression get_env(rcvr), but rcvr could be, and usually is, a prvalue) or always rvalues (e.g. scheduler concept has the expression get_env(schedule(std::forward<Sch>(sch)))).

The intent here was that get_env() is a function that takes as an argument a const T& and thus allows prvalues to bind to it. We basically just want to require that get_env() finds a const-qualified member-function. The use of as_const() does not seem to mirror the semantics of a function with a const T& parameter, so I suggest we change it to something else that expresses the intent.

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after five votes in favour during reflector poll.

This could use the "reified object" idea from 25.3 [range.access].

Proposed resolution:

This wording is relative to N4993.

1. Add to the end of 33.1 [exec.general] as indicated:

~~-?- For a subexpression expr, let AS-CONST(expr) be expression-equivalent to~~

~~[!(const auto& x) noexcept -> const auto& { return x; }](expr)~~

2. Modify 33.5.2 [exec.get.allocator] as indicated:

-1- get_allocator asks a queryable object for its associated allocator.

-2- The name get_allocator denotes a query object. For a subexpression env, get_allocator(env) is expression-equivalent to MANDATE-NOTHROW(~~as_const~~AS-CONST(env).query(get_allocator)).

3. Modify 33.5.3 [exec.get.stop.token] as indicated:

-2- The name get_stop_token denotes a query object. For a subexpression env, get_stop_token(env) is expression-equivalent to:

(2.1) — MANDATE-NOTHROW(~~as_const~~AS-CONST(env).query(get_stop_token)) if that expression is well-formed.

4. Modify 33.5.4 [exec.get.env] as indicated:

-1- execution::get_env is a customization point object. For a subexpression o, execution::get_env(o) is expression-equivalent to:

(1.1) — `MANDATE-NOTHROW(as_const AS_CONST(o).get_env())` if that expression is well-formed.

5. Modify 33.5.5 [\[exec.get.domain\]](#) as indicated:

-2- The name `get_domain` denotes a query object. For a subexpression `env`, `get_domain(env)` is expression-equivalent to `MANDATE-NOTHROW(as_const AS_CONST(env).query(get_domain))`.

6. Modify 33.5.6 [\[exec.get.scheduler\]](#) as indicated:

-2- The name `get_scheduler` denotes a query object. For a subexpression `env`, `get_scheduler(env)` is expression-equivalent to `MANDATE-NOTHROW(as_const AS_CONST(env).query(get_scheduler))`.

7. Modify 33.5.7 [\[exec.get.delegation.scheduler\]](#) as indicated:

-2- The name `get_delegation_scheduler` denotes a query object. For a subexpression `env`, `get_delegation_scheduler(env)` is expression-equivalent to `MANDATE-NOTHROW(as_const AS_CONST(env).query(get_delegation_scheduler))`.

8. Modify 33.5.8 [\[exec.get.fwd.progress\]](#) as indicated:

-2- The name `get_forward_progress_guarantee` denotes a query object. For a subexpression `sch`, let `Sch` be `decltype(sch)`. If `Sch` does not satisfy `scheduler`, `get_forward_progress_guarantee` is ill-formed. Otherwise, `get_forward_progress_guarantee(sch)` is expression-equivalent to:

(2.1) — `MANDATE-NOTHROW(as_const AS_CONST(sch).query(get_forward_progress_guarantee))` if that expression is well-formed.

9. Modify 33.5.9 [\[exec.get.compl.sched\]](#) as indicated:

-2- The name `get_completion_scheduler` denotes a query object template. For a subexpression `q`, the expression `get_completion_scheduler<completion-tag>(q)` is ill-formed if `completion-tag` is not one of `set_value_t`, `set_error_t`, or `set_stopped_t`. Otherwise, `get_completion_scheduler<completion-tag>(q)` is expression-equivalent to

`MANDATE-NOTHROW(as_const AS_CONST(q).query(get_completion_scheduler<completion-tag>))`

4179. Wrong range in [alg.search]

Section: 26.6.15 [\[alg.search\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Oscar Slotosch **Opened:** 2024-12-05 **Last modified:** 2025-02-07

Priority: Not Prioritized

View all other [issues](#) in [\[alg.search\]](#).

Discussion:

Originally reported as editorial request [#7474](#):

During the qualification of the C++ STL Validas has pointed me to the following issue:

The specification of 26.6.15 [\[alg.search\]](#) has a wrong range. Currently the range is `"[first1, last1 - (last2 - first2))"` (exclusive) but should be `"[first1, last1 - (last2 - first2)]"` (inclusive). So please correct the closing `"")` to `"]"`. Otherwise the last occurrence will not be found. We observed the issue in C++14 and C++17 and [cppreference.com](#).

The implementations do the right thing and work correctly and find even the last occurrence.

For example in the list {1, 2, 3, 4, 5} the pattern {4, 5} should be found (obviously). In the case the last element is not included it will not be found.

[Demonstration on godbolt](#) shows that the implementation is working and finds the pattern.

[2024-12-07; Daniel comments and provides wording]

The mentioned wording issue is present in all previous standard versions, including the [SGI STL](#). It needs to be fixed in all search variants specified in 26.6.15 [\[alg.search\]](#) (except for the form using a `Searcher`).

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after ten votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N4993](#).

1. Modify 26.6.15 [alg.search] as indicated:

```
template<class ForwardIterator1, class ForwardIterator2>
constexpr ForwardIterator1
search(ForwardIterator1 first1, ForwardIterator1 last1,
       ForwardIterator2 first2, ForwardIterator2 last2);

[...]
```

```
template<class ExecutionPolicy, class ForwardIterator1, class ForwardIterator2,
        class BinaryPredicate>
ForwardIterator1
search(ExecutionPolicy&& exec,
       ForwardIterator1 first1, ForwardIterator1 last1,
       ForwardIterator2 first2, ForwardIterator2 last2,
       BinaryPredicate pred);
```

-1- *Returns*: The first iterator i in the range $[first1, last1 - (last2 - first2))$ such that for every nonnegative integer n less than $last2 - first2$ the following corresponding conditions hold: $*(i + n) == *(first2 + n)$, $pred(*(i + n), *(first2 + n)) != false$. Returns $first1$ if $[first2, last2)$ is empty, otherwise returns $last1$ if no such iterator is found.

-2- [...]

```
template<forward_iterator I1, sentinel_for<I1> S1, forward_iterator I2,
        sentinel_for<I2> S2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<I1, I2, Pred, Proj1, Proj2>
constexpr subrange<I1>
ranges::search(I1 first1, S1 last1, I2 first2, S2 last2, Pred pred = {},
               Proj1 proj1 = {}, Proj2 proj2 = {});

template<forward_range R1, forward_range R2, class Pred = ranges::equal_to,
        class Proj1 = identity, class Proj2 = identity>
requires indirectly_comparable<iterator_t<R1>, iterator_t<R2>, Pred, Proj1, Proj2>
constexpr borrowed_subrange_t<R1>
ranges::search(R1&& r1, R2&& r2, Pred pred = {},
               Proj1 proj1 = {}, Proj2 proj2 = {});
```

-3- *Returns*:

(3.1) — $\{i, i + (last2 - first2)\}$, where i is the first iterator in the range $[first1, last1 - (last2 - first2))$ such that for every non-negative integer n less than $last2 - first2$ the condition

$bool(invoker(pred, invoker(proj1, *(i + n)), invoker(proj2, *(first2 + n))))$

is true.

(3.2) — Returns $\{last1, last1\}$ if no such iterator exists.

-4- [...]

```
template<class ForwardIterator, class Size, class T = iterator_traits<ForwardIterator>::value_type>
constexpr ForwardIterator
search_n(ForwardIterator first, ForwardIterator last,
         Size count, const T& value);

[...]
```

```
template<class ExecutionPolicy, class ForwardIterator, class Size,
        class T = iterator_traits<ForwardIterator>::value_type,
        class BinaryPredicate>
ForwardIterator
search_n(ExecutionPolicy&& exec,
         ForwardIterator first, ForwardIterator last,
         Size count, const T& value,
         BinaryPredicate pred);
```

-5- [...]

-6- [...]

-7- *Returns*: The first iterator i in the range $[first, last - count)$ such that for every non-negative integer n less than $count$ the condition E is true. Returns $last$ if no such iterator is found.

-8- [...]

```
template<forward_iterator I, sentinel_for<I> S,
        class Pred = ranges::equal_to, class Proj = identity,
        class T = projected_value_t<I, Proj>>
requires indirectly_comparable<I, const T*, Pred, Proj>
constexpr subrange<I>
ranges::search_n(I first, S last, iter_difference_t<I> count,
                 const T& value, Pred pred = {}, Proj proj = {});

template<forward_range R, class Pred = ranges::equal_to,
```

```
class Proj = identity, class T = projected_value_t<iterator_t<R>, Proj>>
requires indirectly_comparable<iterator_t<R>, const T*, Pred, Proj>
constexpr borrowed_subrange_t<R>
ranges::search_n(R&& r, range_difference_t<R> count,
                const T& value, Pred pred = {}, Proj proj = {});
```

-9- Returns: {i, i + count} where i is the first iterator in the range [first, last - count] such that for every non-negative integer n less than count, the following condition holds: invoke(pred, invoke(proj, *(i + n)), value). Returns {last, last} if no such iterator is found.

-10- [...]

4186. regex_traits::transform_primary mistakenly detects typeid of a function

Section: 28.6.6 [\[re.traits\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Jiang An **Opened:** 2024-12-18 **Last modified:** 2025-02-07

Priority: Not Prioritized

View other [active issues](#) in [re.traits].

View all other [issues](#) in [re.traits].

Discussion:

28.6.6 [\[re.traits\]](#)/7 currently says typeid(use_facet<collate<charT>>) == typeid(collate_byname<charT>), which is always false because use_facet<collate<charT>> is a function template specialization while collate_byname<charT> is a class template specialization. This looks like misuse, and has been present in TR1 ([N1836](#)).

Presumably the intended operand is use_facet<collate<charT>>(getloc()).

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 28.6.6 [\[re.traits\]](#) as indicated:

```
template<class ForwardIterator>
string_type transform_primary(ForwardIterator first, ForwardIterator last) const;
```

-7- Effects: If

```
typeid(use_facet<collate<charT>>(getloc())) == typeid(collate_byname<charT>)
```

and the form of the sort key returned by collate_byname<charT>::transform(first, last) is known and can be converted into a primary sort key then returns that key, otherwise returns an empty string.

4189. cache_latest_view should be freestanding

Section: 17.3.2 [\[version.syn\]](#), 25.2 [\[ranges.syn\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Hewill Kang **Opened:** 2024-12-23 **Last modified:** 2025-02-07

Priority: Not Prioritized

View other [active issues](#) in [version.syn].

View all other [issues](#) in [version.syn].

Discussion:

cache_latest_view can be freestanding, but this never comes up in the discussion, which seems to be an oversight.

Previous resolution [SUPERSEDED]:

This wording is relative to [N5001](#).

1. Modify 17.3.2 [\[version.syn\]](#) as indicated:

```
#define __cpp_lib_ranges_cache_latest 202411L // freestanding, also in <ranges>
```

2. Modify 25.2 [\[ranges.syn\]](#) as indicated:

```
#include <compare>           // see 17.11.1 \[compare.syn\]
#include <initializer_list>   // see 17.10.2 \[initializer.list.syn\]
#include <iterator>           // see 24.2 \[iterator.synopsis\]

namespace std::ranges {
    [...]
    // 25.7.34 \[range.cache.latest\], cache latest view
    template<input_range V>
        requires view<V>
        class cache_latest_view; // freestanding

    namespace views { inline constexpr unspecified cache_latest = unspecified; } // freestanding
    [...]
}
```

[2025-01-04; Tim Song suggests alternative wording]

While we are here, we can use the new convention from [P2407](#) to dramatically simplify <ranges>. Most future additions to this header should have no problem being freestanding, so that is the right default.

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after nine votes in favour during reflector poll.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 17.3.2 [\[version.syn\]](#) as indicated:

```
#define __cpp_lib_ranges_cache_latest 202411L // freestanding, also in <ranges>
```

2. Delete all "*freestanding*" comments in 25.2 [\[ranges.syn\]](#), header <ranges> synopsis, and then modify as indicated:

```
// mostly freestanding
#include <compare>           // see 17.11.1 \[compare.syn\]
#include <initializer_list>   // see 17.10.2 \[initializer.list.syn\]
#include <iterator>           // see 24.2 \[iterator.synopsis\]

[...]
namespace std::ranges {
    // 25.5.6 \[range.elementsof\], class template elements_of
    template<range R, class Allocator = allocator<byte>>
        struct elements_of; // hosted
    [...]

    // 25.6.6 \[range.istream\], istream view
    template<movable Val, class CharT, class Traits = char_traits<CharT>>
        requires see below
        class basic_istream_view; // hosted
    template<class Val>
        using istream_view = basic_istream_view<Val, char>; // hosted
    template<class Val>
        using wistream_view = basic_istream_view<Val, wchar_t>; // hosted

    namespace views {
        template<class T> constexpr unspecified istream = unspecified; // hosted
    }
    [...]
}
```

[4191](#). P1467 changed the return type of `pow(complex<float>, int)`

Section: 29.4.10 [\[cmplx.over\]](#) **Status:** [Tentatively Ready](#) **Submitter:** Tim Song **Opened:** 2025-01-10 **Last modified:** 2025-02-07

Priority: Not Prioritized

View other [active issues](#) in [\[cmplx.over\]](#).

View all other [issues](#) in [\[cmplx.over\]](#).

Discussion:

Before C++20, 29.4.10 [cplx.over] says that this produces a `complex<double>`. This was confirmed by LWG 844⁽¹⁾ and consistent with C99.

P1467 changed the return type to `complex<common_type_t<float, int>>`, which is `complex<float>`. This is a breaking change that does not appear to have been intentional.

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after eight votes in favour during reflector poll.

Proposed resolution:

This wording is relative to N5001.

1. Modify 29.4.10 [cplx.over] as indicated:

-3- Function template `pow` has additional constexpr overloads sufficient to ensure, for a call with one argument of type `complex<T1>` and the other argument of type `T2` or `complex<T2>`, both arguments are effectively cast to `complex<common_type_t<T1, T3>>`, where `T3` is double if `T2` is an integer type and `T2` otherwise. If `common_type_t<T1, T3>` is not well-formed, then the program is ill-formed.

4196. Complexity of `inplace_merge()` is incorrect

Section: 26.8.6 [alg.merge] Status: [Tentatively Ready](#) Submitter: Stephen Howe Opened: 2025-01-22 Last modified: 2025-02-07

Priority: 4

View other [active issues](#) in [alg.merge].

View all other [issues](#) in [alg.merge].

Discussion:

For N5001, section 26.8.6 [alg.merge] p5, it says for `merge()` *Complexity* (emphasis mine):

For the overloads with no `ExecutionPolicy`, at most $N - 1$ comparisons and applications of each projection

For N5001, section 26.8.6 [alg.merge] p11, it says from `inplace_merge()` *Complexity* (emphasis mine):

Complexity: Let $N = \text{last} - \text{first}$:

(11.1) — For the overloads with no `ExecutionPolicy`, and if enough additional memory is available, exactly $N - 1$ comparisons.

(11.2) — Otherwise, $\mathcal{O}(N \log N)$ comparisons.

This wording should be (emphasis mine)

Complexity: Let $N = \text{last} - \text{first}$:

(11.1) — For the overloads with no `ExecutionPolicy`, and if enough additional memory is available, at most $N - 1$ comparisons.

(11.2) — Otherwise, $\mathcal{O}(N \log N)$ comparisons.

Consider the 2 sequences in a `std::vector` of ints and assume that `inplace_merge` has enough memory:

{ 1 }, { 2, 3, 4, 5, 6, 7 }

N is 7, 7 elements. So $N - 1$ is 6

If you `inplace_merge()` the two sequences, the 1 is compared with 2 and 1 is output. But now the 1st sequence is over, so the 2nd sequence is copied. Only 1 comparison is done, not 6.

[2025-01-25; Daniel comments]

The [SGI STL archive](#) correctly says "at most" as well.

[2025-02-07; Reflector poll]

Set status to Tentatively Ready after seven votes in favour during reflector poll. There were nine votes for P0 (Tentatively Ready), but also several for NAD Editorial, because 16.3.2.4 [structure.specifications]/7 explicitly says that all complexity requirements are upper bounds and it's conforming to do less work. The consensus was that this is still a clarifying improvement.

Proposed resolution:

This wording is relative to [N5001](#).

1. Modify 26.8.6 [\[alg.merge\]](#) as indicated:

```
template<class BidirectionalIterator>
constexpr void inplace_merge(BidirectionalIterator first,
                             BidirectionalIterator middle,
                             BidirectionalIterator last);
template<class ExecutionPolicy, class BidirectionalIterator>
void inplace_merge(ExecutionPolicy&& exec,
                   BidirectionalIterator first,
                   BidirectionalIterator middle,
                   BidirectionalIterator last);
template<class BidirectionalIterator, class Compare>
constexpr void inplace_merge(BidirectionalIterator first,
                             BidirectionalIterator middle,
                             BidirectionalIterator last, Compare comp);
template<class ExecutionPolicy, class BidirectionalIterator, class Compare>
void inplace_merge(ExecutionPolicy&& exec,
                   BidirectionalIterator first,
                   BidirectionalIterator middle,
                   BidirectionalIterator last, Compare comp);
template<bidirectional_iterator I, sentinel_for<I> S, class Comp = ranges::less,
        class Proj = identity>
requires sortable<I, Comp, Proj>
constexpr I ranges::inplace_merge(I first, I middle, S last, Comp comp = {}, Proj proj = {});
```

-7- [...]

-8- *Preconditions*: [...]

-9- *Effects*: Merges two sorted consecutive ranges [first, middle) and [middle, last), putting the result of the merge into the range [first, last). The resulting range is sorted with respect to comp and proj.

-10- *Returns*: last for the overload in namespace ranges.

-11- *Complexity*: Let $N = \text{last} - \text{first}$:

(11.1) — For the overloads with no ExecutionPolicy, and if enough additional memory is available, ~~exactly~~ **at most** $N - 1$ comparisons.

(11.2) — Otherwise, $\mathcal{O}(N \log N)$ comparisons.

In either case, twice as many projections as comparisons.